



The Agent-Driven Web

A different web for every visitor.

A personalised GitHub Wrapped. The model picks the cards from a typed catalog; React renders them.

THE PRODUCT

A personalised GitHub Wrapped.

Drop a handle. Different profiles get different cards in different orders — not different values in the same template.

- Public profile + repos + recent activity, fetched server-side.
- One AI call returns **analysis** and a **render plan**.
- React renders pre-built typed components from the catalog.



Three pieces. One contract.

1 Profile read

analysis

```
{
  handle: "jrmyyee",
  oneLineSummary: "...",
  creativeProfile: { archetype, palette },
  primaryLanguages: [ ... ],
  greenFlags: [ ... ],
  interestingnessScore: { ... }
}
```

What the model saw. Languages, patterns, strengths — with evidence behind each one. Zod-validated.

2 Render plan

renderPlan.picks[]

```
[
  { component: "HeroQuote",
    variant: "headline",
    size: "wide",
    reason: "single dominant theme" },
  { component: "RepoSpotlight", ... },
  ...
]
```

Show these cards, in this order, because of these reasons. 4–6 typed picks. The reasoning is visible.

3 Component catalog

11 typed React components

HeroQuote	headline
LanguageRing	stats
ActivityHeatmap	activity
RepoSpotlight	project
TopicGraph	topics
MilestoneTimeline	arc
GreenFlagCard	+
RedFlagCard	-
SuperlativeDeck	awards
WorkStyleCard	read
InfluenceMap	reach

Pre-built, typed React components. The model picks IDs from this enum; it never writes JSX.

The render plan can only reference IDs that exist. If a card doesn't have the data behind it, the normaliser drops it. No broken cards because the model got creative.

One typed call. Two surfaces.



Human surface · /v/[handle]
Server Component composes the catalog. Inspector chat is a client component. Lenses derive a new render plan over the same analysis.

Agent surface · /v/[handle]/mcp · /intent
Middleware rewrites Accept: application/intent+json to the JSON route. MCP server (Streamable HTTP) for richer clients.

Three pieces of the SDK.

The AI SDK is broad — **Core, UI, Elements, workflows**. **AI Gateway** sits alongside as a separate product. This project uses three.

SERVER

AI SDK Core

```
const { output } = await generateText({
  model: 'openai/gpt-5.1-instant',
  output: Output.object({ schema }),
  prompt: profilePayload,
});
// → { analysis, renderPlan }
```

One structured call. Schema-constrained. Returns a typed object.

BOUNDARY

AI Gateway

```
// swap models without app changes
model: 'openai/gpt-5.1-instant'
model: 'anthropic/claude-sonnet-4.6'
model: 'openai/gpt-5.4-nano'

// BYOK keeps your provider credits
providerOptions.gateway.byok
```

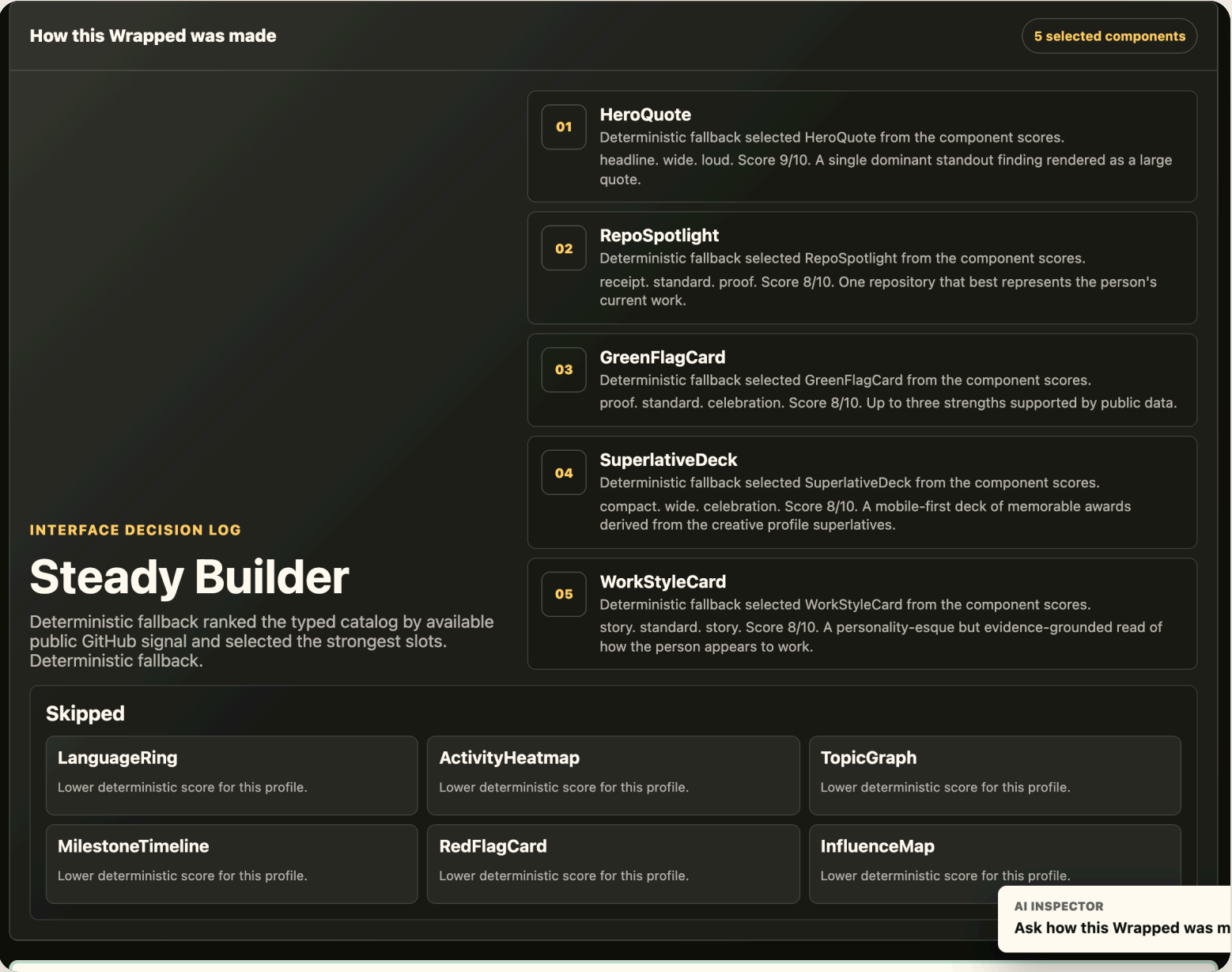
Routing, BYOK, and usage in one place — visible in the Gateway dashboard.

BROWSER

useChat

```
const { messages, sendMessage, status }
= useChat({
  transport: new HttpChatTransport({
    api: '/api/wrapped/chat',
  }),
});
```

Drives the lens redesigns you saw. Streaming, message state, and errors — handled.



THE RECEIPT

The page shows its work.

Every Wrapped includes a decision log — cards picked, cards skipped, and why.

SELECTED 4–6 picks from the 11-card catalog, in order.

SKIPPED Empty data slots and lower scores, each with a reason.

LENS Re-runs the render plan, not the analysis.

Three stages of personalisation.

STAGE 1 · BUILT

Profile-driven

The model drives the personalisation. It picks the story, picks the cards. One structured call, one render plan, one inspectable receipt.

- One AI call returns analysis + render plan
- Decision log on every render

STAGE 2 · NEXT

Intent-driven

The page picks the lens itself based on what the visitor's trying to do. Lenses are cheap because the analysis is durable — intent becomes the input.

- Recruiter / founder / IC lenses
- Page recomposes from observed intent

STAGE 3 · LATER

Persistent agent

An agent that lives longer than one page view. Memory so it knows you. Permissions so it acts safely. Audit trail. A different shape entirely. Not built.

- Memory · permissions · audit
- Acts for the user, not just renders

LIVE

agent-driven- web.vercel.app

/v/<handle>

